
CORSO DI ARCHITETTURE DEGLI ELABORATORI A.A. 2025/2026
PROF.SSA VALENTINA CASOLA

ASSEMBLY ARM

Dr. Alessandra Somma
alessandra.somma@unina.it

GNU ASSEMBLY ARM

- Il **linguaggio assembly** consente di esprimere il programma ad uno stesso livello semantico del repertorio di istruzioni del processore. Introduce convenzioni e strumenti che facilitano la scrittura e la comprensione del programma: le istruzioni sono rappresentate mediante codici mnemonici testuali e le etichette (simboli testuali) possono essere utilizzate per riferirsi a specifiche locazioni di memoria.
- Per una data famiglia di processori possono essere definiti diversi linguaggi assembly. Per i sistemi basati su processori ARM, i linguaggi più utilizzati sono:
 - Il linguaggio supportato dall'assemblatore **armasm** che fa parte della ARM Compiler toolchain, integrata nell'ambiente di sviluppo prodotto dalla società Keil, acquisita nel 2005 da ARM Ltd;
 - Il linguaggio supportato dall'ambiente di sviluppo **IAR Embedded Workbench**;
 - Il linguaggio **ARM GAS**, supportato dall'assemblatore **GNU as** (riferimento del corso)
 - Le convenzioni sintattiche dei linguaggi sono diverse e differenti sono le direttive di assemblaggio → un programma scritto per un assemblatore non potrà essere assemblato senza modifiche da un assemblatore diverso.

FORMATO DELLE ISTRUZIONI ASSEMBLY GAS

- Il **codice sorgente** di un programma assembly consiste in una sequenza di **linee di codice**. Ciascuna linea può essere:
 - Una **linea di commento**: ha solamente lo scopo di fornire informazioni utili a chi legge il codice sorgente, è ignorata dall'assemblatore. Nell'assembly GAS, il **marcatore di inizio commento** è **@**. Se il commento è racchiuso tra i marcatori ***** e ***** può estendersi anche su più linee consecutive.
 - Un'**istruzione**: corrisponde ad istruzioni in linguaggio macchina del processore (la corrispondenza sappiamo che NON è 1:1). L'assemblatore convertirà l'istruzione in linguaggio assembly nella corrispondente codifica binaria delle istruzioni in linguaggio macchina eseguibili dal processore.
 - Una **pseudo-istruzione**: ha una struttura sintattica analoga a quella di un'istruzione. L'assemblatore traduce una pseudo-istruzione in una o più istruzioni consecutive in linguaggio macchina.
 - Una **direttiva di assemblaggio**: ha lo scopo di inserire nel codice sorgente delle indicazioni per l'assemblatore. Ad esempio, possono servire ad indicare all'assemblatore come allocare i dati del programma in memoria.

STRUTTURA DI UN'ISTRUZIONE IN ASSEMBLY GAS

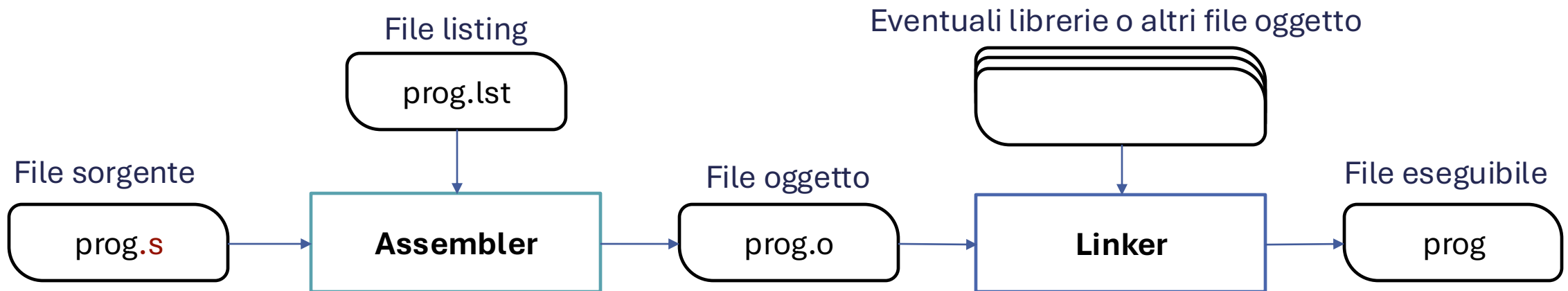
- Un'istruzione **assembly** ha la seguente struttura generale:

label: mnemonic operands @comment

- **label (etichetta)** è opzionale, è una sequenza di lettere maiuscole/minuscole (non può iniziare con una cifra decimale ma può contenerla) e serve ad identificare (in un qualunque altro punto del codice) una specifica istruzione mediante un nome simbolico. In Assembly GAS, il : funge da delimitatore delle etichette, per differenziare una label da un codice mnemonico e consente all'assemblatore di riconoscere correttamente una label anche se preceduta a sinistra da caratteri di spazio.
 - Nel linguaggio GAS, è lecito anche l'uso di una label delimitata dal carattere : da sola su una riga di codice (si intenderà associata all'istruzione posta sulla linea successiva).
- **mnemonic (codice mnemonico)** è una stringa di testo senza spazi che identifica una specifica istruzione del processore.
- **operands (operandi)** è una sequenza di uno, due o tre simboli che indicano dove l'istruzione prende i dati su cui opera.
- **comment (commento)** è una stringa di testo inserita nel codice a scopo di documentazione.

CICLO DI SVILUPPO

- Lo **sviluppo di un programma in linguaggio assembly** (come in qualsiasi linguaggio) è un processo che segue le metodologie dell'ingegneria del software: analisi delle specifiche, scrittura del software e fase di verifica del comportamento corretto.
- La codifica in linguaggio assembly è un processo che consiste dei seguenti passi:



- Produzione con un text editor di un file testuale (**file sorgente**) contenente il codice in linguaggio assembly.
- Assemblaggio**: l'assemblatore traduce ogni istruzione assembly in codice macchina, risolve simboli, etichette e offset, organizza le sezioni codice/dati. Il file .o è un file binario (non è più un file di testo).
- Collegamento (linking) tra uno o più file oggetto per ottenere in output **il file eseguibile** finale.

SIMULATORE ARMSIM#

- L'esecuzione di un programma assembly ARM richiede la disponibilità di un sistema basato su processore ARM. Utilizzeremo il simulatore **ARMSIM#** che integra un assembler ARM Gas ed un linker. Sviluppato dalla University of Victoria è rilasciato come programma **eseguitibile per PC con sistema operativo Windows** (esiste anche la versione per Linux) ed è scaricabile liberamente, per usi didattici:

<https://webhome.cs.uvic.ca/~nigelh/ARMSim-V2.1/Windows/index.html>

- Una volta lanciato in esecuzione ARMSIM#, è possibile caricare nel simulatore il codice sorgente di un programma assembly, scritto secondo la sintassi ARM GAS in un file testuale con estensione .s.
 - La redazione del file sorgente può essere effettuata con qualsiasi editor di testo (ad esempio Blocco Note oppure Notepad).
- ARMSIM# implementa anche le funzioni di un **debugger** per individuare cause di comportamento incorretto del software, consentendo al programmatore di eseguire un'istruzione alla volta o il run fino ad un punto di arresto (breakpoint).

Hexadecimal
Unsigned Decimal
Signed Decimal

R0	:000000ff
R1	:00001150
R2	:00000000
R3	:00000000
R4	:00000000
R5	:00000000
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00001150
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00011400
R14 (lr)	:00001014
R15 (pc)	:00001014

CPSR Register
Negative (N) : 1
Zero (Z) : 0
Carry (C) : 0
Overflow (V) : 0
IRQ Disable : 1
FIQ Disable : 1
Thumb (T) : 0
CPU Mode : System
0x800000df

00001004	EB000034	E3A000FF
0000100C	EB00000C	EB000031
00001014	E59F0120	EB000009
0000101C	EB00002E	E3A00000
00001024	EB000006	EB00002B
0000102C	E3E000FE	EB000003
00001034	EB000028	E3A00018
0000103C	E3A01000	EF123456
00001044	E92D407F	E1A01000
0000104C	E59F20EC	E3510000
00001054	BA000001	E3A0002B
0000105C	EA000000	E3A0002D
00001064	E4C20001	E59F30D4
0000106C	E3A06000	E4934004
00001074	E3540001	BA000022
0000107C	1A000000	E3A06001
00001084	E3A05000	E1A00001
0000108C	E3510000	BA000002
00001094	E0500004	4A000006
0000109C	EA000002	E0800004
000010A4	E3500000	CA000002
000010AC	E1A01000	E2855001
000010B4	EAF000F3	E3550000
000010BC	1A000001	E3560001
000010C4	1AFF00E9	E3A06001
000010CC	E59F0074	E7D00005
000010D4	E4C20001	EAF000E4
000010DC	E92D4001	E3A00001

```
prova.o
@ ***** Code Segment *****
@test segment - printing integers to console

.equ    SWI_Write, 0x05
.equ    SWI_Exit, 0x18

.text

.global print_int           @ Make the label visible

_start:
00001000:E59FA130      ldr    r10, =block
00001004:EB000034      bl     output_crlf

00001008:E3A000FF      ldr    r0, =255
0000100C:EBFFFFFE      bl     print_int
00001010:EB000031      bl     output_crlf

00001014:E59F0120      ldr    r0, =2000000001
00001018:EBFFFFFE      bl     print_int
0000101C:EB00002E      bl     output_crlf

00001020:E3A00000      ldr    r0, =0
00001024:EBFFFFFE      bl     print_int
00001028:EB00002B      bl     output_crlf

0000102C:E3E000FE      ldr    r0, =-255
00001030:EBFFFFFE      bl     print_int
00001034:EB000028      bl     output_crlf

00001038:E3A00018      mov    r0, #SWI_Exit
0000103C:E3A01000      mov    r1, #0
```

0001139C:81818181
000113A0:81818181
000113A4:81818181
000113A8:81818181
000113AC:81818181
000113B0:81818181
000113B4:81818181
000113B8:81818181
000113BC:81818181
000113C0:81818181
000113C4:81818181
000113C8:81818181
000113CC:81818181
000113D0:81818181
000113D4:81818181
000113D8:81818181
000113DC:81818181
000113E0:000000FF
000113E4:00001150
000113E8:00000000
000113EC:00000000
000113F0:00000000
000113F4:00000000
000113F8:000000FF
000113FC:00001014
00011400:????????
00011404:????????
00011408:????????
0001140C:????????
00011410:????????
00011414:????????
00011418:????????
0001141C:????????
00011420:????????
00011424:????????
00011428:????????
0001142C:????????
00011430:????????
00011434:????????
00011438:????????
0001143C:????????
00011440:????????
00011444:????????
00011448:????????
0001144C:????????
00011450:????????
00011454:????????
00011458:????????
0001145C:????????
00011460:????????

Area che mostra
il codice
assembly
caricato nella
macchina
simulata

RegistersView

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	:	000000ff
R1	:	00001150
R2	:	00000000
R3	:	00000000
R4	:	00000000
R5	:	00000000
R6	:	00000000
R7	:	00000000
R8	:	00000000
R9	:	00000000
R10 (s1)	:	00001150
R11 (fp)	:	00000000
R12 (ip)	:	00000000
R13 (sp)	:	00011400
R14 (lr)	:	00001014
R15 (pc)	:	00001014

CPSR Register

Negative (N) : 1

Zero (Z) : 0

Carry (C) : 0

Overflow (V) : 0

IRQ Disable: 1

FIQ Disable: 1

Thumb (T) : 0

CPU Mode : System

0x800000df

MemoryView0

Word Size

8Bit 16Bit 32Bit

00001004	EB000034	E3A000FF
0000100C	EB00000C	EB000031
00001014	E59F0120	EB000009
0000101C	EB00002E	E3A00000
00001024	EB000086	EB00002B
0000102C	E3E000FE	EB000093
00001034	EB000028	E3A00018
0000103C	E3A01000	EF123456
00001044	E92D407F	E1A01000
0000104C	E59F20EC	E3510000
00001054	BA000001	E3A0002B
0000105C	EA000000	E3A0002D
00001064	E4C20001	E59F30D4
0000106C	E3A06000	E4934004
00001074	E3540001	BA000022
0000107C	1A000000	E3A06001
00001084	E3A05000	E1A00001
0000108C	E3510000	BA000002
00001094	E0500004	4A000006
0000109C	EA000002	E0800004
000010A4	E3500000	CA000002
000010AC	E1A01000	E2855001
000010B4	EAF000F3	E3550000
000010BC	1A000001	E3560001
000010C4	1AFF00E9	E3A06001
000010CC	E59F0074	E7D00005
000010D4	E4C20001	EAF000E4
000010DC	E92D4001	E3A00001

CodeView

prova.o

@ ***** Code Segment *****

@test segment - printing integers to console

.equ SWI_Write, 0x05

.equ SWI_Exit, 0x18

.text

.global print_int @ Make the label visible

_start:

00001000:E59FA130 ldr r10, =block

00001004:EB000034 bl output_crlf

00001008:E3A000FF ldr r0, =255

0000100C:EB0000FE bl print_int

00001010:EB000031 bl output_crlf

00001014:E59F0120 ldr r0, =2000000001

00001018:EB0000FE bl print_int

0000101C:EB00002E bl output_crlf

00001020:E3A00000 ldr r0, =0

00001024:EB0000FE bl print_int

00001028:EB00002B bl output_crlf

0000102C:E3E000FE ldr r0, =-255

00001030:EB0000FE bl print_int

00001034:EB000028 bl output_crlf

00001038:E3A00018 mov r0, #SWI_Exit

0000103C:E3A01000 mov r1, #0

StackView

0001139C:81818181
000113A0:81818181
000113A4:81818181
000113A8:81818181
000113AC:81818181
000113B0:81818181
000113B4:81818181
000113B8:81818181
000113BC:81818181
000113C0:81818181
000113C4:81818181
000113C8:81818181
000113CC:81818181
000113D0:81818181
000113D4:81818181
000113D8:81818181
000113DC:81818181
000113E0:000000FF
000113E4:00001150
000113E8:00000000
000113EC:00000000
000113F0:00000000
000113F4:00000000
000113F8:000000FF
000113FC:00001014
00011400:????????
00011404:????????
00011408:????????
0001140C:????????
00011410:????????
00011414:????????
00011418:????????
0001141C:????????
00011420:????????
00011424:????????
00011428:????????
0001142C:????????
00011430:????????
00011434:????????
00011438:????????
0001143C:????????
00011440:????????
00011444:????????
00011448:????????
0001144C:????????
00011450:????????
00011454:????????
00011458:????????
0001145C:????????
00011460:????????

Register View:
area che mostra
il contenuto dei
registri del
processore

R0 :000000ff
R1 :00001150
R2 :00000000
R3 :00000000
R4 :00000000
R5 :00000000
R6 :00000000
R7 :00000000
R8 :00000000
R9 :00000000
R10 (s1):00001150
R11 (fp):00000000
R12 (ip):00000000
R13 (sp):00011400
R14 (lr):00001014
R15 (pc):00001014

CPSR Register
Negative (N) :1
Zero (Z) :0
Carry (C) :0
Overflow (V) :0
IRQ Disable:1
FIQ Disable:1
Thumb (T) :0
CPU Mode :System
0x800000df

Word Size			
8Bit 16Bit 32Bit			
00001004	EB000034	E3A000FF	
0000100C	EB00000C	EB000031	
00001014	E59F0120	EB000009	
0000101C	EB00002E	E3A00000	
00001024	EB000006	EB00002B	
0000102C	E3E000FE	EB000003	
00001034	EB000028	E3A00018	
0000103C	E3A01000	EF123456	
00001044	E92D407F	E1A01000	
0000104C	E59F20EC	E3510000	
00001054	BA000001	E3A0002B	
0000105C	EA000000	E3A0002D	
00001064	E4C20001	E59F30D4	
0000106C	E3A06000	E4934004	
00001074	E3540001	BA000022	
0000107C	1A000000	E3A06001	
00001084	E3A05000	E1A00001	
0000108C	E3510000	BA000002	
00001094	E0500004	4A000006	
0000109C	EA000002	E0800004	
000010A4	E3500000	CA000002	
000010AC	E1A01000	E2855001	
000010B4	EAF000F3	E3550000	
000010BC	1A000001	E3560001	
000010C4	1AFF00E9	E3A06001	
000010CC	E59F0074	E7D00005	
000010D4	E4C20001	EAF000E4	
000010DC	E92D4001	E3A00001	

```
prova.o  
  
@ ***** Code Segment *****  
@test segment - printing integers to console  
  
        .equ    SWI_Write, 0x05  
        .equ    SWI_Exit, 0x18  
  
        .text  
  
        .global print_int          @ Make the label visible  
_start:  
00001000:E59FA130      ldr    r10, =block  
00001004:EB000034      bl     output_crlf  
  
00001008:E3A000FF      ldr    r0, =255  
0000100C:EBFFFFFE      bl     print_int  
00001010:EB000031      bl     output_crlf  
  
00001014:E59F0120      ldr    r0, =2000000001  
00001018:EBFFFFFE      bl     print_int  
0000101C:EB00002E      bl     output_crlf  
  
00001020:E3A00000      ldr    r0, =0  
00001024:EBFFFFFE      bl     print_int  
00001028:EB00002B      bl     output_crlf  
  
0000102C:E3E000FE      ldr    r0, =-255  
00001030:EBFFFFFE      bl     print_int  
00001034:EB000028      bl     output_crlf  
  
00001038:E3A00018      mov    r0, #SWI_Exit  
0000103C:E3A01000      mov    r1, #0
```

0001139C:81818181
000113A0:81818181
000113A4:81818181
000113A8:81818181
000113AC:81818181
000113B0:81818181
000113B4:81818181
000113B8:81818181
000113BC:81818181
000113C0:81818181
000113C4:81818181
000113C8:81818181
000113CC:81818181
000113D0:81818181
000113D4:81818181
000113D8:81818181
000113DC:81818181
000113E0:000000FF
000113E4:00001150
000113E8:00000000
000113EC:00000000
000113F0:00000000
000113F4:00000000
000113F8:000000FF
000113FC:00001014
00011400:????????
00011404:????????
00011408:????????
0001140C:????????
00011410:????????
00011414:????????
00011418:????????
0001141C:????????
00011420:????????
00011424:????????
00011428:????????
0001142C:????????
00011430:????????
00011434:????????
00011438:????????
0001143C:????????
00011440:????????
00011444:????????
00011448:????????
0001144C:????????
00011450:????????
00011454:????????
00011458:????????
0001145C:????????
00011460:????????

Memory View:
area che mostra
il contenuto di
una porzione
della memoria
centrale

SUDDIVISIONE DEL CODICE IN SEGMENTI

- Un file sorgente assembly contiene sia codice che dati. Siccome il processore esegue le istruzioni in ordine sequenziale (a meno che non ci siano istruzioni di salto) *non è corretto inserire in modo arbitrario dati all'interno di una sequenza di istruzioni*. Un codice sorgente tipicamente è strutturato in:
 - Un **segmento di codice**. L'assemblatore assume per convenzione che il file sorgente contenga in testa il segmento codice. Se così non è, il codice sorgente deve contenere obbligatoriamente una indicazione esplicita dell'inizio del segmento codice mediante la direttiva di assemblaggio **.text**. Questa direttiva garantisce che la prima istruzione e quindi anche tutte le successive siano collocate ad un indirizzo multiplo di 4 (allineato sul margine di parola).
 - Un **segmento di dati** il cui inizio è marcato dalla direttiva di assemblaggio **.data**.

ENTRY POINT E TERMINAZIONE DI UN PROGRAMMA

- L'etichetta **_start** è un'**etichetta speciale** a cui per convenzione il linker fa riferimento per individuare l'**entry point** di un programma, ovvero l'indirizzo della prima istruzione da eseguire una volta che il sistema operativo ha caricato il programma in memoria.
- Quando il programma viene caricato in ARMSIM# il simulatore inizializza il registro R15 (PC) del processore al valore dell'indirizzo associato all'etichetta **_start**.

```
_start: instruction_0  
        instruction_1  
        ...
```

SWI 0x11

- La **terminazione del programma** lanciato dall'utente prevede la terminazione ed un «ritorno al sistema operativo» che si occupa di liberare risorse (es. la memoria) usata dal programma durante la sua esecuzione. Questa procedura di ritorno consiste nell'esecuzione di una system call del sistema operativo (chiamata di sistema) che viene lanciata mediante un'istruzione di software interrupt. Nei processori ARM l'**istruzione SWI** si utilizza per lanciare una system call seguita dall'operando che rappresenta il numero della system call, in particolare in ARMSIM# la terminazione avviene con **SWI 0x11**.

TIPI DI OPERANDI DI UN'ISTRUZIONE ASSEMBLY

- Le istruzioni dell'ISA ARM operano con operandi di tre tipi differenti:
 - Operando il cui valore è **costante**, essendo determinato a tempo di assemblaggio.
 - E' rappresentato con il suo valore numerico preceduto dal simbolo del cancelletto # (Vedremo che ci saranno delle limitazioni sul valore costante che è possibile rappresentare).
 - Operandi il cui valore è prelevato da un **registro** del processore.
 - In assembly, l'operando è specificato attraverso il suo nome (e.g. R0, R1 etc). Il codice dell'istruzione contiene l'indice del registro, cioè un numero intero compreso tra 0 e 15 bit codificato su 4 bit.
 - Operandi il cui valore è prelevato dalla **memoria** (previsti solo per istruzioni di trasferimento dati).
 - Il processore deve determinare l'indirizzo effettivo (**effective address - EA**). Questo è possibile in modi diversi, a seconda del **modo di indirizzamento**.
 - Ad esempio un modo di indirizzamento indiretto consiste nello specificare in un registro del processore l'effective address (in assembly si usa l'operatore di parentesi quadre).

OPERANDI DI UN'ISTRUZIONE ASSEMBLY (1)

- Le istruzioni assembly possono avere zero, uno, due o tre operandi espliciti.
 - Un'istruzione può fare riferimento implicitamente ad un operando. Ad esempio l'istruzione BL fa implicitamente uso del registro R14 (LR) nel quale viene salvato il valore corrente del registro R15 (PC).
 - Per molte istruzioni distinguiamo tra **operandi sorgente**, che forniscono dati di input all'istruzione e non vengono alterati, ed **operandi destinazione** sul cui valore l'istruzione produce effetto.
- L'ordine con cui si scrivono gli operandi sorgente e destinazione nella notazione in linguaggio assembly è uno degli elementi rispetto ai quali diversi linguaggi si differenziano.
 - In ARM GAS, l'ordine degli operandi dipende dalla tipologia di istruzione considerata.
 - Per le **istruzioni load e store**, che copiano un dato dalla memoria ad un registro e viceversa, ARM GAS adotta la convenzione di porre **l'operando di tipo registro al primo posto** → nel caso di una LOAD il registro è un operando destinazione, nel caso di una store è sorgente.

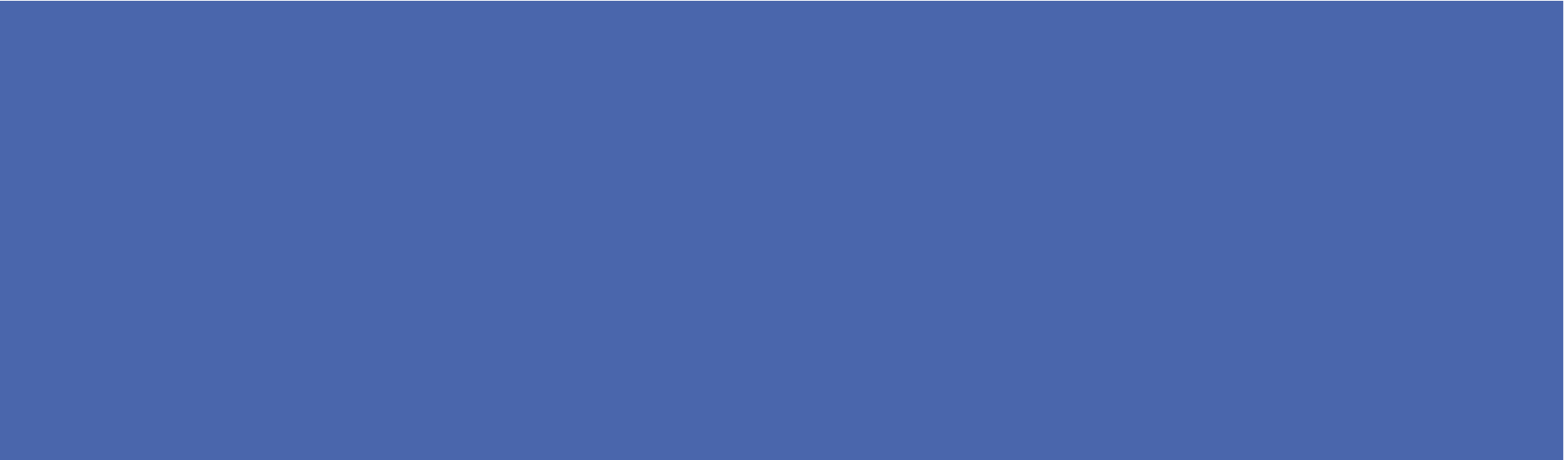
Questo è un modo di indirizzamento detto indiretto che vedremo successivamente

- LDR R2, [R1] → effettua un trasferimento DA memoria A registro: pone il contenuto della locazione di memoria puntata da R1 (operando sorgente) nel registro R2 (operando destinazione).
- STR R2, [R1] → effettua un trasferimento DA registro A memoria: pone il contenuto del registro R2 (operando sorgente) nella locazione puntata da R1 (operando destinazione).

OPERANDI DI UN'ISTRUZIONE ASSEMBLY (2)

- Nelle **istruzioni di elaborazione dei dati** dei processori ARM, l'assembly GAS pone **l'operando destinazione al primo posto**.
 - Ad esempio, `MOV R2, #7` pone il valore costante 7 (operando sorgente) nel registro R2 (operando destinazione). L'istruzione è una move.
 - L'istruzione `ADD R2, R0, R1` ha l'obiettivo di sommare R0 ed R1 (operandi sorgente) e mettere il risultato in R2 (operando destinazione).
- Il terzo operando di una istruzione di elaborazione dati può essere **una costante indicata**, come visto nell'esempio della move, **dal suo valore in decimale preceduto dal cancelletto (#)**. Esistono delle limitazioni sui valori costanti utilizzabili come operando che vedremo.
 - Ad esempio, `ADD R2, R0, #4` esegue la somma di R0 e 4 e pone il risultato in R2.
- Nel caso in cui il valore costante sia **il codice ASCII di un carattere**, l'assembly ARM GAS consente al programmatore di scrivere **il carattere tra apici preceduto dal simbolo #** → sarà l'assemblatore ad inserire in fase di assemblaggio il corrispondente codice ASCII nell'istruzione in linguaggio macchina.
 - Ad esempio, l'istruzione `CMP R3, #'Y'` equivale a `CMP R3, #89`

DIRETTIVE DI ASSEMBLAGGIO IN ASSEMBLY GAS



DIRETTIVE DI ASSEMBLAGGIO (1)

- Le **direttive** servono per **fornire indicazioni all'assemblatore** e l'assembly GAS consente le seguenti:
 - Direttive `.text` e `.data` identificano **l'inizio del segmento di codice e di quello di dati**. L'area codice è di tipo *read-only*, quindi non può contenere dati modificati dal programma.
 - La direttiva `.code 32` o la sua equivalente `.arm` indica all'assemblatore che le istruzioni successive sono riferite **all'instruction set a 32 bit** del processore ARM, mentre la direttiva `.code 16` o la sua equivalente e `.thumb` indica all'assemblatore che le istruzioni successive sono riferite **all'instruction set a 16 bit** Thumb.
 - La direttiva `.end` indica esplicitamente all'assemblatore la **fine del codice sorgente**. Tutto ciò che il programmatore scrive nel programma nelle linee successive a questa direttiva sarà ignorato dall'assemblatore.
 - La direttiva `.include nomefile` comanda all'assemblatore di **effettuare un inserimento nel codice sorgente**, nel punto in cui trova la direttiva `.include`, **di quanto è contenuto nel file nomefile**. Tipicamente, i file inclusi contengono la definizione di macro.

DIRETTIVE DI ASSEMBLAGGIO (2)

- Le **direttive** servono per **fornire indicazioni all'assemblatore** e l'assembly GAS consente le seguenti:
 - La direttiva `.equ symbol, expression` istituisce durante il processo di assemblaggio del file sorgente una nuova entry nella *symbol table*, mediante la quale **associa al simbolo `symbol` il valore corrispondente al risultato dell'espressione costante `expression`**. Ad esempio `.equ seconds, 60*60` assegna al simbolo `seconds` il valore 3600. Ad ogni occorrenza di `seconds` nel codice, l'assemblatore sostituirà 3600.
 - La direttiva `.global symbol` fa in modo che il simbolo `symbol` sia visibile all'esterno del modulo di programma dal linker `ld`.
 - Quando il linker collega due o più moduli oggetto per formare un unico eseguibile, uno stesso simbolo può essere definito in un solo file, ma negli altri file potrebbero esserci riferimenti a quello stesso simbolo.
 - **NOTA:** Il linker `ld` assume che l'entry point di un programma sia associato all'etichetta `_start`. **Il simbolo `_start` deve essere dichiarato mediante la direttiva `.global`.**

DIRETTIVE DI ASSEMBLAGGIO (3)

- Le **direttive** servono per **fornire indicazioni all'assemblatore** e l'assembly GAS consente le seguenti:
 - La direttiva `.extern symbol` informa l'assemblatore as che il simbolo `symbol` è definito in un altro modulo. Il riferimento al simbolo mancante sarà definito a tempo di collegamento dal linker ld.
 - La direttiva `.space number_of_bytes` **riserva in memoria uno spazio di dimensione `number_of_bytes` a partire dall'indirizzo determinato dal valore corrente del PC. L'area di memoria è inizializzata con zeri.**
 - La direttiva `.word value[, value]` **riserva in memoria una o più word inizializzate al valore o ai valori specificati come operando della direttiva. Il termine **word** indica **4 byte**, cioè **quattro locazioni consecutive di memoria**.** Questa direttiva sarà utile per inizializzare gli elementi di un array di numeri interi.
 - La direttiva `.align [number [,padding]]` **incrementa il PC del più piccolo valore che lo renda un multiplo di **2*number**.** La direttiva `.align` equivale a `.align 2` producendo l'allineamento del PC al più piccolo multiplo di 4. L'argomento opzionale di `padding` consente di specificare i valori con cui vanno riempite le locazioni di memoria saltate nell'allineamento.

DIRETTIVE DI ASSEMBLAGGIO (4)

- Le **direttive** servono per **fornire indicazioni all'assemblatore** e l'assembly GAS consente le seguenti:
 - La direttiva `symbol .req Rn` **associa al registro Rn il nome simbolico equivalente symbol** (un alias). L'assembler sostituirà nel codice tutte le occorrenze di symbol con il nome del registro Rn. La direttiva `.unreq symbol` **cancella l'associazione** tra il nome simbolico symbol ed il corrispondente registro Rn.
 - La direttiva `.macro` **indica l'inizio di una macro**, ovvero di *una sequenza di istruzioni a cui viene dato un nome*. Nel codice sorgente, tutte le volte che l'assemblatore trova il nome della macro sostituisce ad esso la sequenza di istruzioni della macro. La direttiva `.endm` delimita la fine della macro.
 - La direttiva `.lorg` indica all'assemblatore un punto del codice nel quale è possibile inserire un pool di costanti, da usarsi in combinazioni con l'istruzione LDR (vedremo più avanti che questa è anche una pseudoistruzione).

UN PRIMO ESEMPIO (1)

```
.global _start
_start: MOV R1, #3
        MOV R2, #4
        ADD R3, R1, R2
        SWI 0x11
```

Carichiamo il codice sorgente (.s) nel simulatore ARMSIM#.
Questa operazione ha l'effetto di assemblare e linkare (nella finestra vediamo il file .o che è in realtà già linkato e pronto all'uso).

The screenshot displays the ARMSIM# simulator interface with three main windows:

- RegistersView:** Shows the state of registers R0 through R15 and the CPSR register. R0-R9 are at 0x00000000, R10 (s1) is 0x00000000, R11 (fp) is 0x00000000, R12 (ip) is 0x00000000, R13 (sp) is 0x0011400, R14 (lr) is 0x00000000, and R15 (pc) is 0x0001000. CPSR shows Negative (N) as 0, Zero (Z) as 0, Carry (C) as 0, Overflow (V) as 0, IRQ Disable as 1, FIQ Disable as 1, Thumb (T) as 0, and CPU Mode as System.
- CodeView:** Displays the assembly code for 'primoesempio.o'. The code is:

```
@ esempio: primo esempio di codice in assembly GAS
@ -----
.global _start
_start: MOV R1, #3
        MOV R2, #4
        ADD R3, R1, R2
        SWI 0x11...
```

The instruction 'MOV R1, #3' is highlighted.
- MemoryView:** Shows the memory layout of the linked object file. The address range 00001000 to 000010F0 is displayed, with corresponding hex values for each instruction.

UN PRIMO ESEMPIO (2)

```
.global _start
_start: MOV R1, #3
        MOV R2, #4
        ADD R3, R1, R2
        SWI 0x11
```

Come abbiamo detto, la direttiva `.global _start` ha l'obiettivo rendere l'etichetta `_start` visibile al linker che carica nel PC (R15) l'indirizzo dell'entry point, ovvero l'indirizzo della prima istruzione del programma. Infatti PC punta a `00001000` e se apriamo la memoria nella locazione puntata leggiamo `E3A01003` che la codifica binaria della prima istruzione.

The screenshot displays a debugger interface with three main panels:

- RegistersView:** Shows the state of various registers. R15 (pc) is highlighted with a red box and contains the value `00001000`.
- CodeView:** Shows the assembly code for `primoesempio.o`. The first instruction, `_start: MOV R1, #3`, is highlighted in blue. A red box highlights the address `00001000:E3A01003` next to this instruction.
- MemoryView0:** Shows the memory contents starting at address `00001000`. The first word at this address is `E3A01003`, which is highlighted with a red box. This value represents the binary encoding of the first instruction.

Red arrows indicate the flow of information: from the assembly code instruction to the memory location, and from the PC register to the memory location.

UN PRIMO ESEMPIO

Questo tasto consente di avanzare nell'esecuzione del programma un'istruzione alla volta

```
.global _start
_start: MOV R1, #3
        MOV R2, #4
        ADD R3, R1, R2
        SWI 0x11
```

1. L'esecuzione della prima istruzione ha avuto l'effetto di:

- Spostare il valore costante 3 nel registro R1
- Incrementare il PC alla prossima istruzione (PC+4) poiché come si vede dal CPSR per il momento la Thumb è disabilitata e quindi le istruzioni sono codificate su 32 bit.

The screenshot displays a debugger interface with three main panels:

- RegistersView:** Shows the state of various registers. R1 is highlighted with a green box and contains the value 0x00000003. R15 (PC) is highlighted with a red box and contains the value 0x00010004. The CPSR Register shows Thumb (T) as 0.
- CodeView:** Shows the assembly code for 'primoesempio.o'. The first instruction, 'MOV R1, #3', is highlighted with a green box. The address 0x00010004 is also highlighted.
- MemoryView0:** Shows the memory contents at address 0x00010000. The instruction at address 0x00010004 is highlighted with a red box and contains the value 0xE3A02004.

UN PRIMO ESEMPIO (4)

```
global _start
_start: MOV R1, #3
        MOV R2, #4
        ADD R3, R1, R2
        SWI 0x11
```

2. L'esecuzione della seconda istruzione ha avuto l'effetto di:

- Spostare il valore costante 4 nel registro R2 (destinazione)
- Incrementare il PC alla prossima istruzione (PC+4)

The screenshot displays a debugger interface with three main panes: RegistersView, CodeView, and MemoryView.

RegistersView: Shows the state of various registers. R2 is highlighted with a green box and contains the value 00000004. R15 (PC) is highlighted with a red box and contains the value 00001008.

CodeView: Shows the assembly code for 'primoesempio.o'. The second instruction, 'MOV R2, #4', is highlighted with a green box. The third instruction, 'ADD R3, R1, R2', is also highlighted.

MemoryView: Shows the memory contents starting at address 00001000. The instruction at address E0813002 is highlighted with a red box.

Arrows indicate the flow of execution: from the assembly code in CodeView to the registers in RegistersView, and from the instruction in MemoryView to the PC register in RegistersView.

UN PRIMO ESEMPIO (5)

_start: .global _start
 MOV R1, #3
 MOV R2, #4
 ADD R3, R1, R2
 SWI 0x11

3. L'esecuzione della terza istruzione ha avuto l'effetto di:

- Sommare il contenuto di R1 e R2 che sono operandi sorgenti
- Copiare il risultato in R3 (operando destinazione)
- Incrementare il PC alla prossima istruzione (PC+4)

The screenshot displays the ARMSIM# debugger interface. The **RegistersView** window shows the state of the registers. R3 is highlighted with a green box and contains the value 0x00000007. R15 (PC) is highlighted with a red box and contains the value 0x0001000C. The **CodeView** window shows the assembly code for the program. The third instruction, `ADD R3, R1, R2`, is highlighted with a green box. The **MemoryView0** window shows the memory at address 0x0001000C, which contains the instruction 0xEF000011, highlighted with a red box. A red arrow points from the R15 (PC) register to the memory address 0x0001000C.

Si fa notare che il PC è sempre allineato ad un multiplo di 4 byte (sul limite di parola) per cui gli ultimi suoi bit sono sempre 00 (pc[1:0]=00). In ARMSIM# il valore mostrato è esadecimale, quindi PC = 0x0000100C = 0000 0000 0000 0000 0001 0000 0000 1100 →
pc[1:0] = 00.

UN PRIMO ESEMPIO (6)

```
.global _start
_start: MOV R1, #3
        MOV R2, #4
        ADD R3, R1, R2
        SWI 0x11
```

4. L'esecuzione dell'ultima e quarta istruzione ha l'effetto di terminare il programma attraverso una system call (SWI 0x11). Quando il processore esegue questa istruzione, ARMSIM# cattura il valore della system call (0x11) che la identifica, interrompe l'esecuzione del codice utente, chiama la routine interna del simulatore per terminare il programma e ferma la simulazione.

The screenshot displays the ARMSIM# simulator interface with three main windows:

- RegistersView:** Shows the state of various registers. R15 (pc) is highlighted with a red box and labeled as 0x0001001c.
- CodeView:** Shows the assembly code for 'primoesempio.o'. The instruction 'SWI 0x11...' is highlighted with a green box.
- MemoryView0:** Shows the memory contents. The address 0x0001001c is selected, and the value 0x81 is shown in a red box, indicating the system call.

Come si può vedere, il simulatore mette il PC all'indirizzo 0x0001001C (un indirizzo «dummy»). Nella locazione puntata, il simulatore inserisce 0x81 per indica «esecuzione terminata». La CPU reale non esegue questa istruzione, ma è il simulatore che mette il PC in una «zona morta», per terminare la simulazione.

ESEMPIO DIRETTIVA ALIGN

@ uso della direttiva .align

.data

a: .byte 0x11

.align 2, 0xFF @incrementa PC di tre

@ per rendere multiplo di $2*2 = 4$ byte, riempiendo con FF

b: .byte 0x22

c: .byte 0x33

d: .byte 0x44

.align @incrementa PC di 1 per rendere multiplo di 4 byte

@ il byte conterra 00

e: .byte 0x55

.align 1

@incrementa PC di 1 per rendere multiplo di 2 byte

f: .byte 0x66

.align 2, 'a'

@ incrementa PC di 1 per rendere multiplo di 4 byte